Informe Proyecto 1

Departamento de Ingeniería de Sistemas y Computación

ISIS-1226 Diseño y Programación Orientada a Objetos

Juan David Vasquez Leyva - 202426062

David Hortua - 202510382

Juan Felipe Leon - 202510397

Universidad de los Andes

Parte de Análisis. **3**
1. El problema que se resuelve **3**
2. Análisis **3**
3. Modelo **4**
3.1 Usuario 4
3.2 Tienda de Juegos 4
3.3 Cafe 5
3.4 Factura 5
3.5 Admin e Informe 5
4. Diagrama de clases **6**
5. Requerimientos Funcionales 7
Lista de excepciones 7
● Requerimientos del Cliente 8
● Requerimientos del Empleado 10
● Requerimientos del Administrador 11
● Requerimientos de consulta (Administrador) 17
Parte de Diseño. **20**
1. Introducción 20
2. Cambios al modelo de dominio **20**
3. Decisiones de diseño del inventario de juegos 22
4. Decisiones de diseño de la calculadora 23
5. Decisiones de diseño de persistencia 24
6. Estructura de paquetes **25**
1. Paquete de Generals 26
2. Paquete de Calculadora 27
3. Paquete de Persistencia 28
4. Paquete de DataBase, Cafe y TiendaDeJuegos 29

Parte de Análisis.

1. El problema que se resuelve

El proyecto tiene como propósito resolver el problema de El Board Game Cafe “Dulces n Dados” el cual es un establecimiento que combina una cafetería que, a su vez, funciona como un lugar para poder jugar juegos de mesa. El negocio requiere un sistema de información que permita gestionar de manera eficiente sus operaciones, abarcando tres grandes áreas: la gestión del inventario de juegos de mesa (préstamo y venta), el funcionamiento del café (empleados, menú y facturas) y las tareas administrativas. Este documento presenta el modelo, los requerimientos funcionales y las restricciones del sistema.

2. Análisis

A partir del análisis del caso de estudio se llegó a los siguientes elementos. El sistema de “Dulces n Dados” se divide en tres grandes categorías: el servicio de préstamo de juegos y reserva de mesas, el servicio de cafetería y restaurante, y el rol del administrador del local. Además, el sistema también cuenta con tres tipos de usuario: Cliente, Empleado (Cocinero o Mesero) y Administrador, cada uno con su login y contraseña.

Con respecto al servicio de préstamo de juegos, para hacer una préstamo de un juego el cliente debe tener una mesa reservada, en la que se especifique el número de personas y si hay menores de edad, verificando que no pase la capacidad máxima del negocio. Existen tres restricciones adicionales según el tipo de juego: los juegos marcados como difíciles necesitan que haya un mesero capacitado para enseñar las reglas. Además, el café maneja un inventario de juegos a la venta, los cuales son distintos a los que están para préstamo, y tanto los clientes como los empleados pueden tenerlos, estos últimos tienen un código de descuento propio.

En cuanto a la cafetería, el menú incluye productos de pastelería y bebida, que pueden ser alcohólicas o calientes. La venta de algunos productos no está permitida dependiendo de los integrantes de la mesa. Por ejemplo, si en la mesa hay niños, entonces no se puede entregar bebidas alcohólicas. También, las bebidas calientes no pueden servirse cuando se está jugando un juego de acción. Todas las ventas tienen impuestos y además le dan punto de fidelidad al cliente y esto puede usarse como descuentos en compras.

Es importante mencionar que los empleados tienen turnos específicos que se repiten cada semana. El sistema debe garantizar que siempre haya al menos 1 cocinero y 2 meseros al mismo tiempo.

Por último, el administrador tiene acceso a la información de todos los juegos que hay en los inventarios de préstamo y de venta. Puede hacer pedidos de juegos nuevos para ambos inventarios, o reparar juegos. También, tiene un historial completo de todos los préstamos del local y su información. Además, tiene que hacer un informe con todos los datos de los juegos y café, incluyendo fechas, horas, montos, etc.

3. Modelo

El modelo se construyó identificando las entidades principales, sus atributos y las relaciones entre ellas. A continuación se presenta una descripción de cada clase y sus características. En esto nos basamos para hacer el UML.

3.1 Usuario

La clase Usuario es abstracta y tiene las siguientes subclases.

***Usuario (abstract)* :** login : String, contraseña : String.
***Cliente* :** idCliente : String, juegosComprados : List<JuegoDeMesa>, juegosFavoritos : List<JuegoDeMesa>, juegosReservados (maximo 2), puntosDeFidelidad : float, mesa : Mesa.
***Administrador* :** Hereda usuario y no tiene más atributos propios.
***Empleado (abstract)* :** turnoLaboral : List<String>, codigoDescuento : String, juegosFavoritos : List<JuegosDeMesa>. Está clase va a ser la superclase de Cocinero y Mesero.
***Cocinero (hereda Empleado)* :** Hereda empleado y tiene mínimo un turno.
***Mesero (hereda Empleado)* :** juegosDificiles : List<JuegoDeMesa>. Debe tener mínimo 2 turnos.

3.2 Tienda de Juegos

Gestiona el inventario de juegos para préstamo y venta.

***TiendaDeJuegos* :** facturas : List<Factura>, inventarioJuegos : InventarioJuegos
***InventarioJuegos* :** juegosPrestamo : List<JuegosDeMesa>, juegosVenta : List<JuegosDeMesaVenta>, historialPrestamos : List<Prestamo>
***JuegoDeMesa* :** idJuego : String, enUso : boolean, añoPublicacion : datetime, empresaProduccion : String, tipoDeJuego : String, minJugadores : int, maxJugadores : int, edadMinima : int, categoria (Cartas / Tablero / Acción) : String, estado : String, dificil : boolean.
***Prestamo* :** idJuego : String, idMesa : String, fechaInicio : datetime, fechaFin : datetime, cliente : String.

3.3 Cafe

Gestiona el funcionamiento de la cafetería.

***Cafe* :** mesa : List<Mesa>, capacidadMaxClientes : int, empleados : List<Empleado>, menuProductos : List<Producto>, facturas : List <Factura>
***Mesa* :** idMesa : String, cantidadClientes : int, tieneNinos : boolean, tieneJovenes : boolean, cliente : Cliente.
***Turno* :** diaSemana : datetime, empleado : Empleado.
***Producto (abtract)* :** nombre : String, precio : float. Esta clase es la superclase de Bebida y Pasteleria.
***Bebida (hereda Producto)* :** alcoholica : boolean, caliente : boolean.
***Pasteleria (hereda Producto)* :** alergenos : List<String>

3.4 Factura

Gestiona los registros de las ventas.

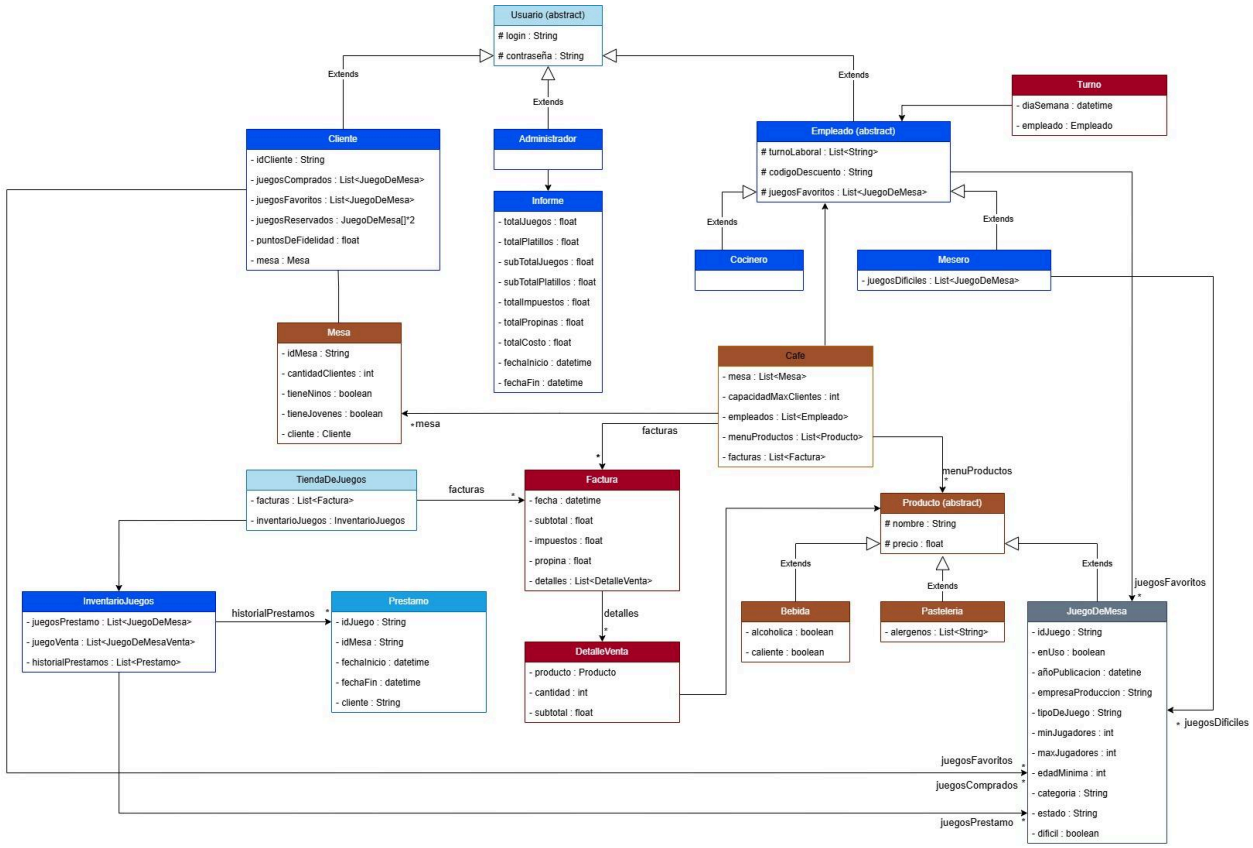
***Factura* :** fecha : datetime,subtotal : float, impuestos : float, propina : float, detalles : List<DetalleVenta>
***DetalleVenta* :** producto : Producto, cantidad : int, subtotal : float.

3.5 Admin e Informe

La clase Administrador debe hacer un informe que debe contener lo siguiente.

***Informe* :** totalJuegos : float, totalPlatillos : float, subTotalJuegos : float, subTotalPlatillos : float, totalImpuestos : float, totalPropinas : float, totalCosto : float, fechaInicio : datetime, fechaFin : datetime.

4. Diagrama de clases



En este diagrama de clases en UML se ve representado el sistema de “Dulces n Dados”. Se pueden apreciar las clases de Empleado, Administrador, y Cliente, las cuales heredan a Usuario. Esto se debe a que todos los usuarios sin importar su tipo, requieren login y contraseña, y tenerlos como una extensión de Usuario evita la repetición de código.

La clase Empleado es abstracta y tiene como subclases a Mesero y Cocinero. El mesero debe conocer juegos difíciles, mientras que el cocinero es un empleado común y corriente. Por su parte, el Administrador tiene un historial completo de los préstamos realizados en el local, junto con su información, además, debe hacer informes detallados de las ventas.

El cliente se relaciona con la mesa a través de la reserva y con los juegos de mesas por medio de la compra o pedirlos prestados. Además, al tener una mesa, puede hacer uso de las funciones de la cafetería, como ordenar productos tanto de pastelería como bebidas. Es importante mencionar que, tanto las bebidas, los productos de pastelería y los juegos de mesa son herederos de la clase Producto.Cada vez que se hace una compra, al cliente se le genera una factura por cada compra de juegos y cada gasto en el cafe.

Finalmente, es importante mencionar que los inventarios de préstamo y de venta se separaron dentro de la clase InventarioJuegos para mantener la distinción entre ambos, ya que el texto menciona que ambas tienen listas completamente diferentes.

5. Requerimientos Funcionales

A continuación se presentan los requerimientos funcionales del sistema, organizados por el actor principal que los ejecuta. Para cada requerimiento se describe su propósito, las entradas que recibe, la salida que produce y las excepciones que puede lanzar.

Lista de excepciones

El sistema utiliza las siguientes excepciones personalizadas para señalar condiciones de error de forma precisa:

- **EntidadNoEncontradaException :** Se lanza cuando un objeto buscado por ID no existe en el sistema.
- **CapacidadExcedidaException :** Se lanza cuando se intenta agregar más personas de las que permite la capacidad del café.
- **JuegoNoDisponibleException :** Se lanza cuando se intenta prestar un juego que ya está en uso.
- **RestriccionEdadException :** Se lanza cuando el juego solicitado no es apto para la edad de los comensales de la mesa.
- **RestriccionJugadoresException :** Se lanza cuando el número de personas en la mesa está fuera del rango de jugadores del juego.
- **ConflictoHorarioException :** Se lanza cuando la asignación o cambio de turno viola el mínimo de empleados requerido.
- **OperacionNoPermitidaException :** Se lanza cuando un actor intenta realizar una acción para la que no tiene permisos.
- **InventarioInsuficienteException :** Se lanza cuando no hay suficientes unidades de un juego en el inventario.
- **SolicitudNoValidaException :** Se lanza cuando una solicitud de cambio de turno o de informe tiene parámetros inválidos.

- **EstadoInvalidoException :** Se lanza cuando se intenta realizar una operación sobre un objeto que no está en el estado adecuado.

Requerimientos del Cliente

reservarJuego

Actor: Cliente

Descripción: Permite a un cliente solicitar el préstamo de un juego de mesa para su mesa. El sistema verifica que el juego esté disponible, que sea apto para el número de personas y las edades presentes en la mesa, y que la capacidad del café no se haya superado. Retorna un objeto Prestamo que registra el inicio del préstamo.

Entradas:

- **clienteId:** String
- **juegoId:** String
- **mesaId:** String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException
- CapacidadExcedidaException

solicitarJuegoPrestado

Actor: Cliente

Descripción: Variante de reservarJuego que también permite a un cliente solicitar un juego prestado para su mesa. Al igual que reservarJuego, valida disponibilidad, restricciones de edad y de número de jugadores antes de registrar el préstamo.

Entradas:

- **clienteId:** String
- **mesaId:** String

- **juegoId:** String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException

comprarJuegoCliente

Actor: Cliente

Descripción: Permite a un cliente comprar uno o varios juegos del inventario de venta en una sola transacción. El sistema verifica que haya suficientes unidades disponibles, aplica el IVA del 19% y genera una factura detallada. Si el cliente tiene un código de descuento válido, este se aplica antes del cálculo de impuestos.

Entradas:

- **clienteId:** String
- **listaJuegos:** List<JuegoDeMesa>

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException

agregarPuntosDeFidelidad hecho

Actor: Cliente (automático tras cada compra)

Descripción: Calcula y acumula los puntos de fidelidad del cliente tras cada compra. El sistema agrega el 1% del monto de la compra como puntos, donde cada punto equivale a un peso y puede ser usado como descuento en compras futuras.

Entradas:

- **clienteId:** String
- **montoCompra:** float

Salida:

- float (puntos agregados)

Excepciones:

- EntidadNoEncontradaException

● Requerimientos del Empleado

solicitarCambioHorario hecho

Actor: Empleado

Descripción: Permite a un empleado solicitar el cambio de uno de sus turnos semanales a otro día disponible. El sistema verifica que el cambio no deje al café con menos del mínimo requerido de cocineros (1) o meseros (2) en ningún turno afectado. La solicitud queda en estado pendiente hasta que el administrador la apruebe o rechace.

Entradas:

- **empleadoId:** String
- **turnoActual:** Turno
- **turnoNuevo:** Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

solicitarIntercambioHorario hecho

Actor: Empleado

Descripción: Permite a dos empleados intercambiar sus turnos entre sí. El sistema verifica que el intercambio no viole los mínimos de cobertura del café en ninguno de los días involucrados. La solicitud queda pendiente hasta la aprobación del administrador.

Entradas:

- **empleado1Id:** String
- **empleado2Id:** String
- **turno1:** Turno
- **turno2:** Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

<u>sugerirPlatillo hecho</u>

Actor: Empleado

Descripción: Permite a cualquier empleado (cocinero o mesero) proponer un nuevo producto para el menú del café. La sugerencia queda registrada en el sistema a la espera de que el administrador la evalúe y decida si la incorpora al menú.

Entradas:

- **empleadoId:** String
- **producto:** Producto

Salida:

- void

Excepciones:

- OperacionNoPermitidaException

● Requerimientos del Administrador

<u>repararJuego</u>

Actor: Administrador

Descripción: Permite al administrador registrar la reparación de un juego del inventario de préstamo. El estado del juego debe ser distinto de 'Nuevo' para poder repararse. Internamente, se reemplaza la copia dañada por una del inventario de ventas y se actualiza el estado del juego.

Entradas:

- **juegoId:** String

Salida:

- void

Excepciones:

- EstadoInvalidoException
- EntidadNoEncontradaException

darJuegoPorRobado

Actor: Administrador

Descripción: Permite al administrador marcar un juego como desaparecido cuando no ha sido devuelto tras un período razonable. El juego se elimina del inventario activo y su estado se actualiza a 'Robado', quedando registrado en el historial de préstamos.

Entradas:

- **juegoId:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

venderJuego

Actor: Administrador

Descripción: Permite al administrador registrar la venta directa de un juego a un cliente desde el inventario de venta. El sistema verifica que haya stock disponible, aplica los impuestos correspondientes y genera una factura.

Entradas:

- **clienteId:** String
- **juegoId:** String
- **cantidad:** int

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException
- EntidadNoEncontradaException

asignarHorario

Actor: Administrador

Descripción: Permite al administrador asignar un nuevo turno a un empleado. El sistema verifica que la asignación no genere conflictos de cobertura mínima del café antes de confirmarla.

Entradas:

- **empleadoId:** String
- **turno:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

asignarTurnoHorario

Actor: Administrador

Descripción: Variante de asignarHorario que permite al administrador establecer el turno semanal de un empleado específico. Valida las mismas restricciones de cobertura mínima.

Entradas:

- **empleadoId:** String
- **turno:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

modificarTurnoHorario

Actor: Administrador

Descripción: Permite al administrador modificar el turno existente de un empleado, reemplazando el turno antiguo por uno nuevo. El sistema valida que el cambio no deje al café sin la cobertura mínima requerida en ninguno de los días involucrados.

Entradas:

- **empleadoId:** String
- **turnoAntiguo:** Turno
- **turnoNuevo:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

aprobarSolicitudHorario

Actor: Administrador

Descripción: Permite al administrador revisar y aprobar una solicitud de cambio o intercambio de turno previamente enviada por un empleado. Una vez aprobada, el sistema actualiza los turnos correspondientes y notifica a los empleados involucrados.

Entradas:

- **administradorId:** String
- **solicitudId:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

aceptarSugerenciaPlatillo

Actor: Administrador

Descripción: Permite al administrador revisar una sugerencia de platillo enviada por un empleado y aprobarla para que sea incorporada al menú del café.

Entradas:

- **administradorId:** String
- **producto:** Producto

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

agregarPlatillo

Actor: Administrador

Descripción: Permite al administrador agregar directamente un nuevo producto al menú del café, sin necesidad de que provenga de una sugerencia. El producto debe tener un estado válido para poder ser incorporado.

Entradas:

- **producto:** Producto

Salida:

- void

Excepciones:

- EstadoInvalidoException

moverJuegoDeVentasAPrestamo

Actor: Administrador

Descripción: Permite al administrador mover un juego del inventario de ventas al inventario de préstamos. El sistema verifica que haya unidades suficientes en el inventario de ventas antes de realizar el movimiento.

Entradas:

- **juegoId:** String

Salida:

- void

Excepciones:

- InventarioInsuficienteException

<u>reabastecerInventarioVentas</u>

Actor: Administrador

Descripción: Permite al administrador comprar y agregar nuevas unidades de un juego al inventario de ventas. El juego debe estar en un estado válido para poder ser reabastecido.

Entradas:

- **juegoId:** String
- **cantidad:** int

Salida:

- void

Excepciones:

- EstadoInvalidoException

<u>reabastecerInventarioPrestamo</u>

Actor: Administrador

Descripción: Permite al administrador comprar y agregar nuevas unidades de un juego al inventario de préstamo. Al igual que con el inventario de ventas, el juego debe estar en un estado válido.

Entradas:

- **juegoId:** String
- **cantidad:** int

Salida:

- void

Excepciones:

- EstadoInvalidoException

reportarJuegoRobado

Actor: Administrador

Descripción: Permite al administrador reportar formalmente un juego como robado. El juego se marca como desaparecido en su estado y queda registrado en el historial de préstamos para efectos contables e históricos.

Entradas:

- **juegoId:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

● Requerimientos de consulta (Administrador)

consultarEstadoJuego

Actor: Administrador

Descripción: Permite al administrador consultar el estado actual de un juego específico del inventario (Nuevo, Bueno, Falta una pieza, Robado, etc.). Esta información es exclusiva del administrador y no es visible para los clientes.

Entradas:

- **juegoId:** String

Salida:

- String (estado actual del juego)

Excepciones:

- EntidadNoEncontradaException

consultarNumPrestamosJuego

Actor: Administrador

Descripción: Permite al administrador consultar cuántas veces ha sido prestado un juego específico a lo largo de toda la historia del café. Es útil para identificar los juegos más populares y tomar decisiones de inventario.

Entradas:

- **juegoId:** String

Salida:

- int (número total de préstamos)

Excepciones:

- EntidadNoEncontradaException

consultarFechaPrestamoJuego

Actor: Administrador

Descripción: Permite al administrador consultar todas las fechas en las que un juego específico ha sido prestado. Retorna una lista de fechas para tener el historial completo de uso del juego.

Entradas:

- **juegoId:** String

Salida:

- List<datetime> (fechas de préstamo)

Excepciones:

- EntidadNoEncontradaException

generarInforme

Actor: Administrador

Descripción: Permite al administrador generar un informe financiero detallado del café para un período de tiempo determinado. El informe incluye el total de ventas de juegos y de cafetería, separados por subtotal, impuestos y propinas, con granularidad diaria, semanal o mensual según el rango de fechas indicado.

Entradas:

- **fechaInicio:** datetime
- **fechaFin:** datetime

Salida:

- Informe

Excepciones:

- SolicitudNoValidaException

Parte de Diseño.

1. Introducción

Este documento registra todas las decisiones de diseño tomadas durante la implementación del sistema Dulces & Dados, comparando el modelo de análisis original (Entrega 1) con el código y UML actuales. Para cada decisión se indica que cambio respecto al análisis inicial y por qué se tomó esa decisión.

2. Cambios al modelo de dominio

Durante la implementación se identificaron varias diferencias respecto al modelo planteado en la Entrega 1. A continuación se muestra el nuevo modelo en UML, y se documenta cada cambio con su justificación.

D1. JuegoDeMesa se separo en JuegoDeMesa y JuegoDeMesaFisico

Cambio: El analisis original tenia una sola clase JuegoDeMesa con atributos como estado y enUso. En el codigo actual existen dos clases: JuegoDeMesa (datos del juego como concepto: nombre, precio, minJugadores, dificil) y JuegoDeMesaFisico (extiende JuegoDeMesa y agrega estadoDelJuego y prestado) que representa una copia fisica especifica.

Justificacion: El enunciado distingue entre la informacion del juego visible para clientes y el estado de cada copia fisica visible solo para el administrador. Una sola clase mezclaba ambas responsabilidades. Con la separacion, JuegoDeMesa es el catalogo publico y JuegoDeMesaFisico es la copia real que puede estar prestada, danada o robada.

D2. JuegoDeMesa extiende Producto del paquete cafe

Cambio: En el analisis, JuegoDeMesa y Producto eran entidades separadas sin relacion de herencia. En el codigo actual, JuegoDeMesa extiende Producto, compartiendo nombre y precio.

Justificacion: Esto permite que DetalleVenta pueda referenciar cualquier tipo de producto (bebida, pasteleria o juego de mesa) con un solo tipo, unificando el sistema de facturacion para todos los tipos de venta sin necesitar clases de detalle separadas.

D3. categoria de JuegoDeMesa reemplazada por TipoDeJuego como enum

Cambio: El analisis tenia categoria : String con valores Cartas, Tablero o Accion. En el codigo existe un enum TipoDeJuego con esos mismos valores como constantes tipadas.

Justificacion: Usar un enum evita errores por erratas al escribir el nombre de la categoria como String (por ejemplo, escribir 'accion' en lugar de 'Accion'). El compilador detecta valores invalidos en tiempo de compilacion, haciendo el codigo mas robusto.

D4. estado de JuegoDeMesaFisico quedo en JuegoDeMesaFisico, no en JuegoDeMesa

Cambio: El analisis tenia estado : String en JuegoDeMesa. En el codigo actual, estado se llama estadoDelJuego y vive en JuegoDeMesaFisico.

Justificacion: El estado (Nuevo, Bueno, Falta una pieza, Robado) describe una copia fisica especifica, no el juego en abstracto. Moverlo a JuegoDeMesaFisico es mas correcto conceptualmente: el juego Catan como concepto no tiene estado, pero la copia numero 3 del Catan si puede estar danada.

D5. Cliente usa JuegoDeMesaFisico en lugar de JuegoDeMesa

Cambio: El analisis tenia juegosComprados : List<JuegoDeMesa> y juegosReservados en Cliente. En el codigo actual, Cliente tiene juegosComprados : ArrayList<JuegoDeMesaFisico> y juegosPrestados : ArrayList<JuegoDeMesaFisico>.

Justificacion: Un cliente no compra o toma prestado el concepto abstracto de un juego, sino una copia fisica especifica. Usar JuegoDeMesaFisico permite saber exactamente que copia tiene el cliente y cual es su estado, lo cual es necesario para el historial del administrador.

D6. Mesa agrego productosOrdenados y tieneBebidaCaliente

Cambio: El analisis no incluia estos atributos en Mesa. En el codigo actual Mesa tiene productosOrdenados : ArrayList<Producto> que registra los productos pedidos durante la visita, y tieneBebidaCaliente : boolean para validar la restriccion con juegos de Accion.

Justificacion: La mesa necesita saber que productos se han ordenado para que GestorVentaCafe pueda calcular el subtotal de la factura directamente desde la mesa. El flag tieneBebidaCaliente es necesario para la restriccion del enunciado: no se puede prestar un juego de Accion si hay bebida caliente en la mesa, y viceversa.

D7. Cafe renombro facturas por catalogoPlatillos y agrego agregarMesa

Cambio: El analisis tenia facturas : List<Factura> en Cafe. En el codigo actual Cafe tiene catalogoPlatillos : ArrayList<Producto> para el menu, y las facturas se movieron a DataBase. Ademas, Cafe tiene el metodo agregarMesa() que crea y agrega mesas internamente.

Justificacion: El catalogo de platillos y las facturas son responsabilidades distintas. Las facturas son datos contables compartidos con la tienda de juegos, por lo que viven en DataBase. El

metodo agregarMesa en Cafe encapsula la creacion de mesas: nadie de afuera modifica la lista directamente, lo que respeta el principio de encapsulamiento.

D8. Turno cambio de tener un solo Empleado a tener listas de Meseros y Cocineros

Cambio: El analisis tenia Turno con un solo atributo empleado : Empleado. En el codigo actual Turno tiene meseros : ArrayList<Mesero> y cocineros : ArrayList<Cocinero>, y su constructor valida que haya minimo 2 meseros y 1 cocinero al crearlo.

Justificacion: El enunciado dice que siempre debe haber minimo 1 cocinero y 2 meseros simultaneamente. Un turno con un solo empleado no podia representar esa restriccion. Con listas separadas por tipo, el turno puede validar los minimos en el constructor y en los metodos agregarMesero, eliminarMesero, agregarCocinero y eliminarCocinero, garantizando la restriccion en todo momento.

D9. DiaSemana se implemento como enum

Cambio: El analisis tenia diaSemana : datetime en Turno. En el codigo actual existe un enum DiaSemana con los valores LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO.

Justificacion: Un turno es recurrente semanal, no una fecha especifica. Usar datetime implicaria inventar una fecha arbitraria para algo que conceptualmente no tiene fecha. El enum representa exactamente el dia de la semana sin fecha, y DataBase usa un array de 7 posiciones (una por dia) indexado por este enum.

D10. DataBase centraliza facturas, usuarios, turnos, solicitudes e informes

Cambio: El analisis no tenia una clase DataBase. Las facturas vivian en Cafe y en TiendaDeJuegos por separado. En el codigo actual existe DataBase con facturas, usuarios, turnos (Turno[7]), solicitudes e informes, compartida entre los modulos.

Justificacion: Las facturas, usuarios y turnos son datos compartidos entre el modulo de cafe y el de tienda de juegos. Tenerlos separados en cada modulo generaria duplicacion e inconsistencias. DataBase es el contenedor unico de los datos persistentes del sistema que trascienden un solo modulo.

D11. Turnos en DataBase como array de 7 posiciones

Cambio: El analisis no especificaba como se almacenaban los turnos. En el codigo actual DataBase tiene turnos : Turno[7], un array de exactamente 7 posiciones, una por cada dia de la semana.

Justificacion: Los dias de la semana son exactamente 7 y no cambian. Un array de tamano fijo es mas eficiente que una lista para este caso porque el acceso por indice es O(1) y no hay

necesidad de buscar por nombre de dia. Cada posicion del array corresponde a un dia segun DiaSemana.

D12. Solicitud como clase abstracta con jerarquia de subclases

Cambio: El analisis mencionaba solicitudes de cambio e intercambio de turno y sugerencias de platillos sin modelarlas como clases. En el codigo existen: Solicitud (abstracta), SolicitudTurno, SolicitudCambioTurno (extiende SolicitudTurno), SolicitudIntercambioTurno y SolicitudSugerenciaPlatillo.

Justificacion: El enunciado describe tres flujos de aprobacion distintos pero con el mismo ciclo de vida: solicitar, quedar pendiente, aprobar o rechazar. Modelarlos como subclases de una clase comun permite tratarlos de forma uniforme en DataBase y en el flujo de aprobacion del administrador. Ademas, Empleado tiene los metodos solicitarCambioHorario y solicitarIntercambioHorario que crean las solicitudes directamente.

3. Decisiones de diseño del inventario de juegos

D13. InventarioJuegos usa HashMap<String, ArrayList<JuegoDeMesaFisico>>

Cambio: El analisis tenia juegosPrestamo : List<JuegoDeMesa> y juegosVenta : List<JuegoDeMesa> como listas planas. En el codigo actual InventarioJuegos tiene inventarioPrestamo y inventarioVenta como HashMap<String, ArrayList<JuegoDeMesaFisico>>, donde la llave es el nombre del juego y el valor es la lista de copias fisicas.

Justificacion: Con una lista plana no hay forma eficiente de saber cuantas copias hay de un juego especifico. Con el HashMap, obtener todas las copias de Catan es simplemente inventarioPrestamo.get('Catan'), y contar las disponibles es .size() en O(1). Ademas, agregar nuevas copias con computeIfAbsent evita verificar si la llave existe antes de agregar.

D14. InventarioJuegos agrego catalogoJuegos como HashMap<String, JuegoDeMesa>

Cambio: El analisis no tenia un catalogo separado. En el codigo actual InventarioJuegos tiene catalogoJuegos : HashMap<String, JuegoDeMesa> que mapea el nombre del juego a su informacion conceptual.

Justificacion: Los inventarios de prestamo y venta contienen copias fisicas (JuegoDeMesaFisico), pero para mostrar el catalogo al cliente se necesita la informacion conceptual del juego (nombre, descripcion, reglas) sin exponer el estado de cada copia. El catalogo permite separar la vista del cliente de los detalles del inventario.

D15. TipoInventario como enum para diferenciar operaciones

Cambio: El analisis no tenia este concepto. En el codigo actual existe el enum TipoInventario con valores VENTAS y PRESTAMO, usado como parametro en validarDisponibilidadJuegos, eliminarJuegoInventario, eliminarPrimerJuego y agregarJuegoInventario.

Justificacion: Sin este enum, habria que duplicar cada metodo del inventario o usar un String como parametro. El enum garantiza que solo se puedan usar los valores validos, detecta errores en compilacion y hace el codigo mas legible. Un metodo como validarDisponibilidadJuegos(nombre, cantidad, TipoInventario.VENTAS) es autoexplicativo.

D16. InventarioJuegos valida disponibilidad antes de eliminar

Cambio: El analisis no detallaba este flujo. En el codigo actual, antes de cualquier eliminacion del inventario se llama a validarDisponibilidadJuegos que lanza JuegoNoDisponibleException si la cantidad solicitada supera el stock. Luego eliminarJuegoInventario hace la eliminacion buscando por idJuego especifico, y eliminarPrimerJuego quita la primera copia disponible.

Justificacion: Separar la validacion de la eliminacion respeta el principio de responsabilidad unica. La validacion protege el inventario de operaciones invalidas antes de modificarlo. Tener dos metodos de eliminacion (por ID especifico vs primer disponible) permite manejar los casos de reparacion de un juego especifico y prestamo del primero disponible de forma distinta.

4. Decisiones de diseño de la calculadora

D17. Calculadora como interfaz con constantes y metodos default

Cambio: El analisis no tenia esta clase. En el codigo actual existe la interfaz Calculadora con las constantes IVA=0.19, impuestoConsumoAsociado=0.08, descuentoEmpleados=0.2, descuentoClientes=0.1 y propinaBase=0.1. Tiene metodos default para calcularDescuento, calcularPropina, calcularPuntosFidelidad y calcularSubtotal, y un metodo abstracto calcularImpuestos.

Justificacion: Centralizar las operaciones matematicas en una interfaz evita duplicar logica entre GestorVentaJuegos y GestorVentaCafe. Los metodos default permiten que las implementaciones hereden el comportamiento comun sin repetir codigo. Solo calcularImpuestos es abstracto porque es la unica operacion que difiere entre los dos tipos de venta.

D18. CalculadoraVentaJuegos y CalculadoraVentaCafe como implementaciones separadas

Cambio: El analisis no tenia estas clases. En el codigo actual CalculadoraVentaJuegos implementa calcularImpuestos con IVA del 19%, y CalculadoraVentaCafe implementa calcularImpuestos con impuesto al consumo del 8%. CalculadoraVentaCafe agrega ademas calcularTotal(Mesa mesa, double propina).

Justificacion: El enunciado establece impuestos distintos por tipo de venta. Dos clases concretas permiten que cada gestor instancie su propia calculadora sin saber que formula usa el otro. El metodo calcularTotal en CalculadoraVentaCafe encapsula el calculo completo de una venta de cafeteria desde la mesa.

5. Decisiones de diseño de persistencia

D19. Serializacion Java con tres archivos separados por modulo

Cambio: El analisis solo mencionaba que la informacion debe ser persistente en archivos. En el codigo actual existe un sistema de persistencia con tres interfaces (IPersistenciaCafe, IPersistenciaTiendaDeJuegos, IPersistenciaDataBase) y tres implementaciones de serializacion Java, coordinadas por CentralPersistencia.

Justificacion: La serializacion Java serializa el grafo completo de objetos recursivamente, lo que permite guardar y cargar el estado completo del sistema con pocas lineas. Separar en tres archivos por modulo permite que si solo cambia el cafe, solo se persiste ese archivo. CentralPersistencia actua como fabrica, desacoplando el mecanismo de persistencia del resto del sistema.

D20. Todas las clases del dominio implementan Serializable

Cambio: El analisis no mencionaba Serializable. En el codigo actual Cafe implementa Serializable con serialVersionUID = 1L. Para que la serializacion funcione correctamente, todas las clases referenciadas (Mesa, Producto, Bebida, Pasteleria, Empleado, Turno, etc.) tambien deben implementar Serializable.

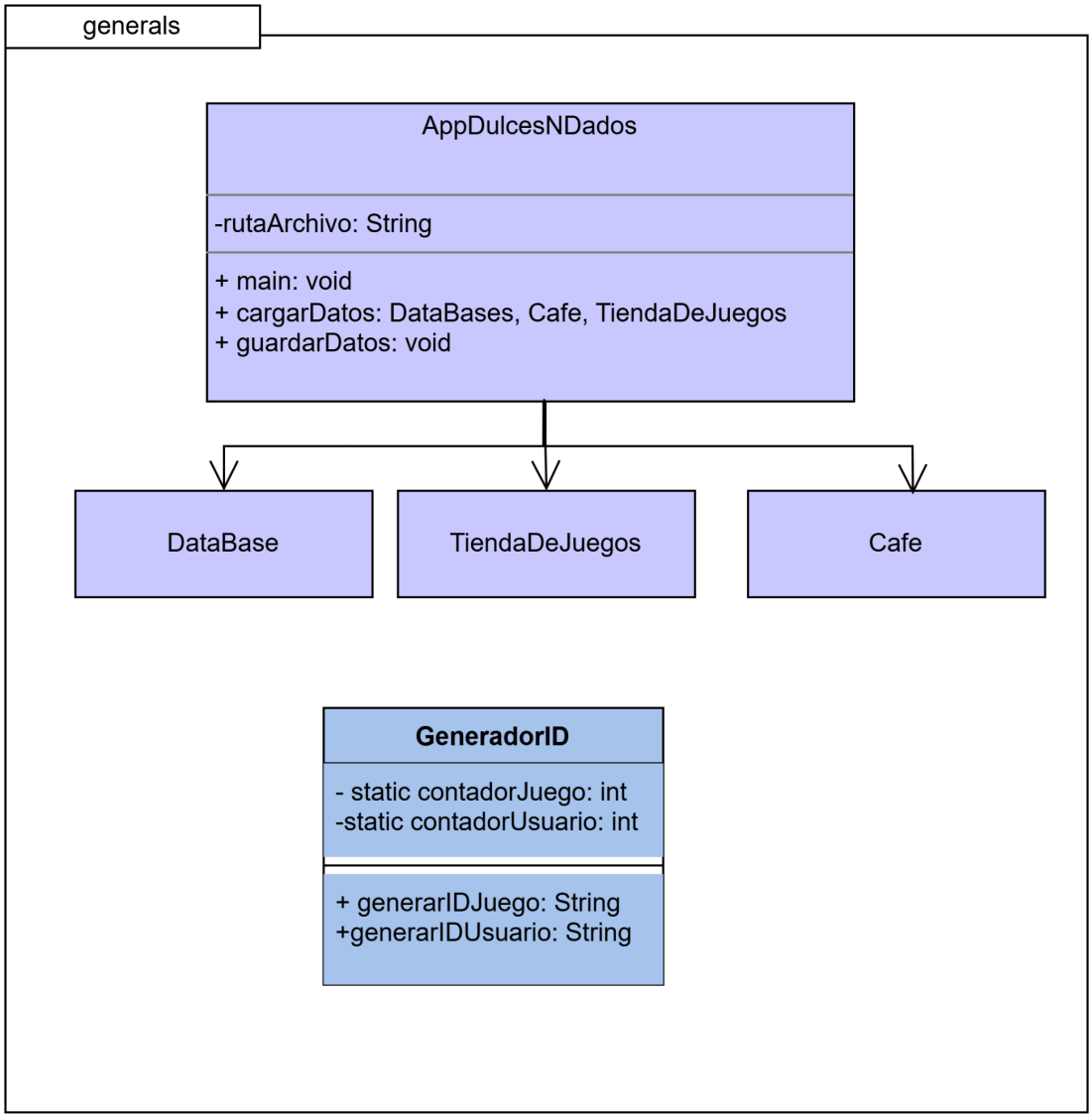
Justificacion: La serializacion Java falla con NotSerializableException si alguna clase en el grafo de objetos no implementa Serializable. El serialVersionUID garantiza compatibilidad entre versiones: si se cambia la clase, se incrementa el numero para que Java detecte archivos guardados con versiones anteriores.

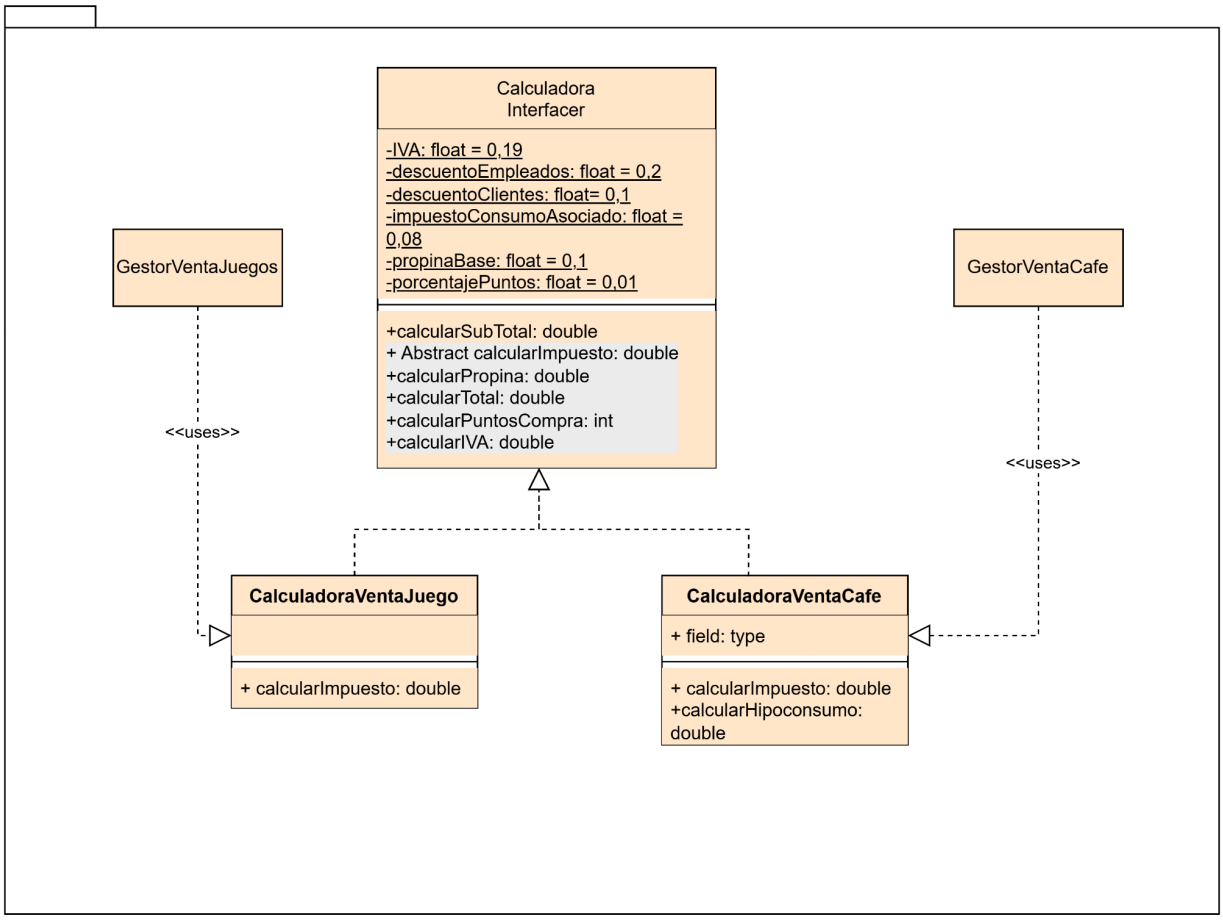
6. Estructura de paquetes

El sistema se organizó en los siguientes paquetes, cada uno con una responsabilidad clara que no estaba explícitamente definida en el análisis original:

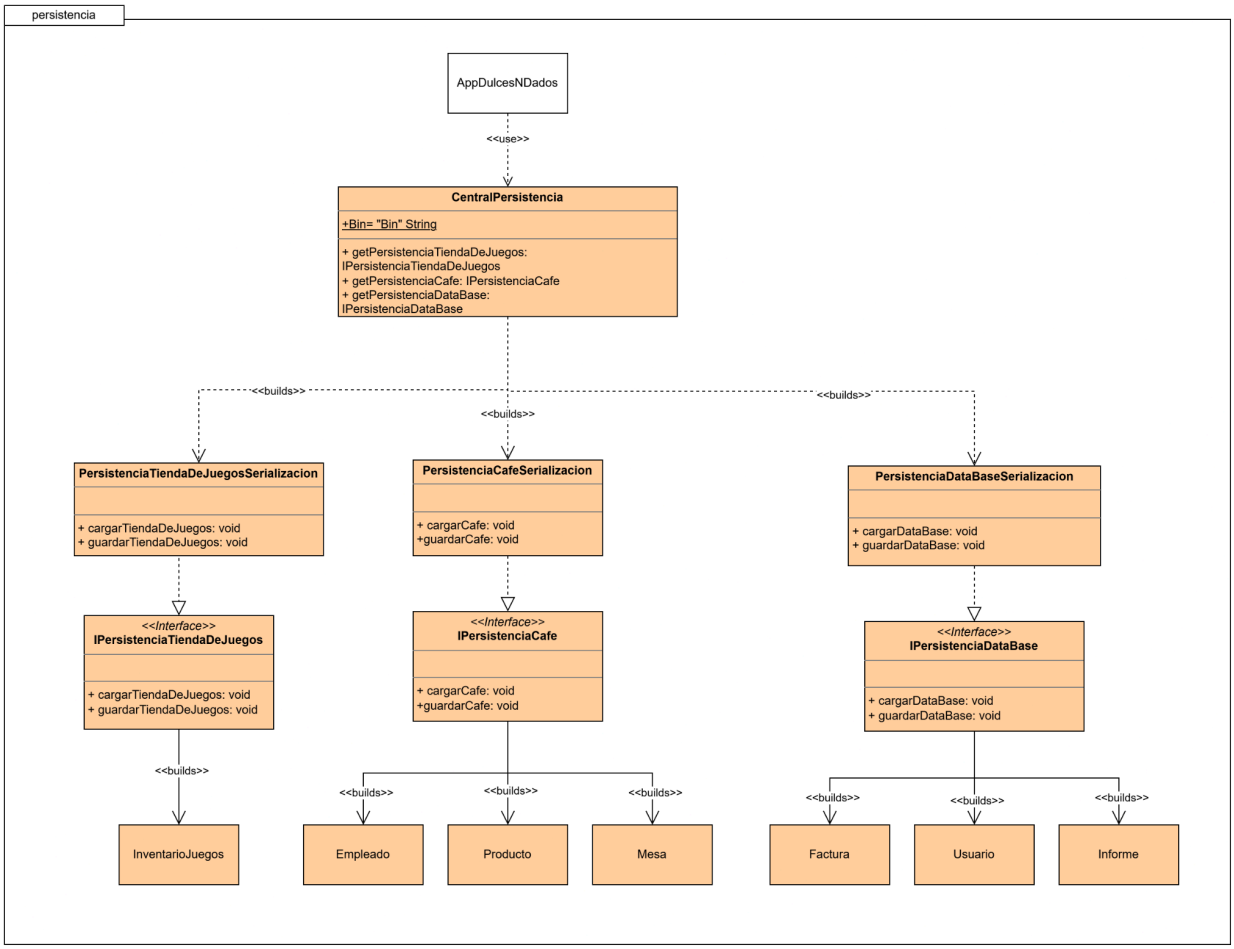
- cafe — Clases del dominio del cafe: Cafe, Mesa, Producto, Bebida, Pasteleria, GestorVentaCafe.
- tiendaDeJuegos — Inventario y ventas: TiendaDeJuegos, InventarioJuegos, JuegoDeMesa, JuegoDeMesaFisico, GestorVentaJuegos, GestorInventarioJuegos, Prestamo, TipoDeJuego, TipoInventario, TipoVenta.
- dataBase — Datos compartidos: DataBase, Usuario, Cliente, Empleado, Mesero, Cocinero, Factura, DetalleVenta, Turno, DiaSemana, Solicitud y sus subclases, Informe.
- calculadora — Interfaz Calculadora, CalculadoraVentaJuegos, CalculadoraVentaCafe.
- persistencia — CentralPersistencia, interfaces y clases de serializacion.
- exceptions — Todas las excepciones personalizadas del sistema.
- generals — DulcesNDados (punto de entrada), generadorID.

1. Paquete de Generals

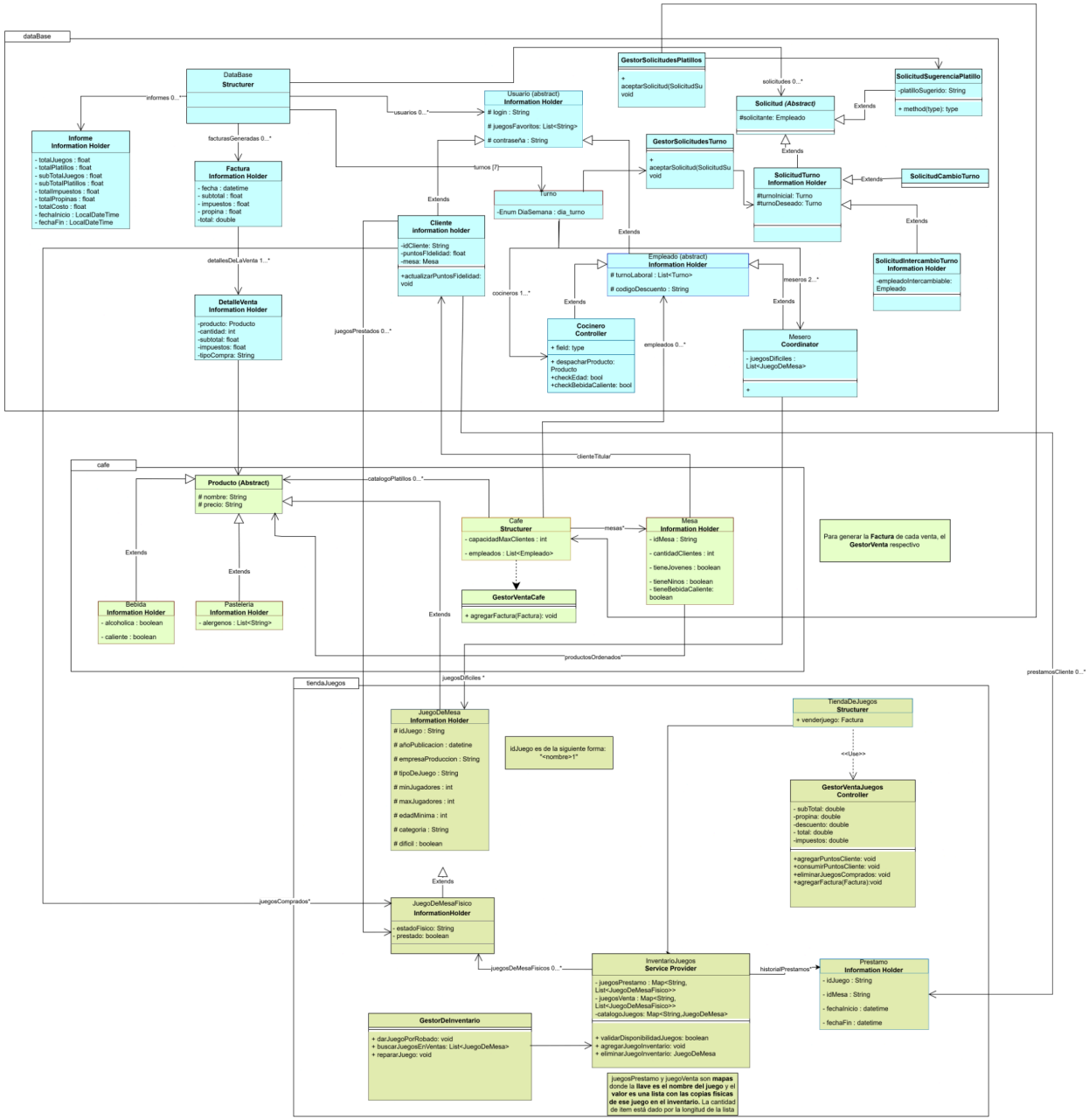




2. Paquete de Calculadora



3. Paquete de Persistencia



4. Paquete de DataBase, Cafe y TiendaDeJuegos